Project context and goals

What we are building

E-Waste Compute Grid reuses voluntarily contributed, functional retired laptops for independent computing batches. A coordinator stores jobs, divides them into tasks, and lets laptop agents claim work, run an approved workload, and return validated results. The first real workload is MobileNetV2 ImageNet classification: one class and confidence per image. It is not object detection. The dummy workload produces synthetic results to test the pipeline.

Why it fits waste and energy

The defensible waste claim is productive reuse of genuinely retired hardware. Record device history, consent and useful completed work. Count physical devices once; call their mass hardware repurposed unless disposal was actually avoided. Reuse does not automatically save electricity. The dashboard currently reports assumed baseline power multiplied by accepted execution time, not measured savings. Compare identical useful work and a declared measurement boundary before making an energy-saving claim.

Team and working style

Kushagra: backend and API contracts. Vaibhav: worker, Docker and installation. Nikhil: workload data, benchmarks and environmental evidence. Chinmay: dashboard and user experience. Harry means mentoring/architecture; Sejal means implementation/debugging. Priorities: correctness, maintainability, security, architecture, performance and scalability. Inspect existing code, trace root causes, avoid unnecessary rewrites, and distinguish tested behavior from plans.

Where this handoff applies

Snapshot: 9 October 2026. Source workspace: D:/E-waste/E-waste. Paths in this document are relative to the repository so teammates can use their own checkout. This supplied directory has no canonical Git history or verified clean-clone release. Recover the team repository and agree on integration branches; do not invent a repository URL or treat the historical ZIP as the new installer.

Architecture and verified progress

Execution flow

Dashboard -> HTTP API -> PostgreSQL jobs/tasks -> laptop worker -> Docker workload -> signed result upload -> validated completion -> downloadable aggregated JSON. Workers contact the API outbound. LAN IP discovery and incoming laptop firewall ports are not required for pairing. AWS can host the coordinator/storage, but this handoff does not verify or update the temporary AWS deployment.

Existing engineering foundation

PostgreSQL task claims use FOR UPDATE SKIP LOCKED. Leases identify individual attempts; heartbeats renew valid leases; retries have a budget; stale attempts cannot overwrite accepted work. Completion and metric insertion are duplicate-safe. Image output must account for every assigned filename exactly once as a prediction or error. There is no Redis broker or verified energy-optimized scheduler.

Verified before onboarding

Phase 1: 57 tests passed at that checkpoint; genuine photos and model/label hashes are pinned; native and restricted Docker inference were exercised. Phase 2: browser upload of ten photos, chunk size five, completed two Docker tasks with exact downloaded result accounting on the development machine. Job 2a34f804-a3ea-4ead-80bf-6888157f0408. This is not evidence of a separate retired laptop or a three-device recovery demonstration.

Onboarding step 1

Completed: expiring pairing sessions, administrator invitation, owner decision, one worker identity across duplicate approvals, saved-identity reconnect and revocation. Verification: 27 backend tests with zero skips, including 20 real PostgreSQL/HTTP cases; 16 worker tests; frontend build/lint. The existing demo retained one node, one job, two tasks and two metrics after its additive schema upgrade.

Onboarding step 2

Source preview implemented; interactive approval/stop acceptance remains pending. See the verification section for tested scope.

Meaning of hardware values

CPU cores are logical processors reported by the OS. RAM is total usable system RAM, not memory consumed by the grid. Online capacity is not a live utilization meter. IDLE means no grid task assigned; WORKING includes transfer, execution and reporting. Docker defaults cap each workload at two CPU equivalents and 2 GiB; limits are not a promise of actual consumption.

The pairing protocol teammates must preserve

Create and invite

POST /api/pairing: send name, cpu_cores, ram_gb, os, capabilities and baseline_power_w. The response includes pairing_id, an eight-character case-sensitive pairing_code, pairing_secret, grid_name, worker_name, state and expires_at. Only the code is shared. POST /api/pairing/invite: the dashboard uses its admin bearer token and sends {pairing_code}. Invitation changes WAITING to INVITED; it does not create a worker.

Owner decision

GET /api/pairing/:id uses the private pairing bearer token and returns state without secrets. POST /api/pairing/:id/decision uses that same token and sends {decision: approve} or {decision: decline}. Approval returns node_id, node_secret, enrollment_mode and heartbeat_interval_seconds. Save the identity before starting computation. Decline creates no node. A decision before invitation returns 409. Expired decisions return 410.

Duplicate and lost-response behavior

Concurrent approvals create one Node. Retrying the same approved decision recovers the same identity while the session is valid. Retrying decline remains declined. Changing an accepted decision returns 409. Revocation blocks credential recovery. A lost response is not evidence that the server rejected approval; retry the existing session instead of generating another identity.

Reconnect and revoke

GET /api/nodes/session validates an existing identity; POST also refreshes hardware/capabilities. Both need bearer node_secret and x-node-id and return no secret. POST /api/nodes/:node_id/revoke accepts the admin token or that node's credentials. The agent stops on authentication denial instead of silently re-enrolling. Active work stops when the agent observes revocation; unfinished assignments recover through lease timeout.

State and credentials

WAITING -> INVITED -> APPROVED or DECLINED. Unfinished sessions expire after ten minutes by default. Codes contain 48 random bits; new private credentials contain 256. The database stores pairing code/secret hashes. Approved sessions are retained as consent evidence. Expired unapproved sessions are swept after 24 hours. Never put private tokens in URLs, screenshots, bundle files or ordinary logs.

Step-by-step remaining implementation

Step 2 - invitation and owner approval

Kushagra + Vaibhav + Chinmay. The pairing client, dashboard invitation form and desktop window are now present in source. Finish interactive approval/stop/restart acceptance before declaring the step complete. Keep network/preflight work off the UI thread. Save approved credentials atomically, recover a lost response without another node, and never use a global admin/enrollment token on the laptop. Deliver a source-based end-to-end flow before packaging.

Step 3 - participation and resource policy

Kushagra + Vaibhav. Validate and persist owner-selected CPU/RAM limits, enforce them in Docker, and make task admission respect effective capacity. Define pause as no new tasks while current work finishes; define stop as safe cancellation. Preserve pause on restart. Test concurrent assignment versus pause/revoke and exact once-only result acceptance.

Step 4 - desktop reliability and credential protection

Vaibhav + Chinmay. Complete responsive UI, connection state, errors/logs, settings persistence, protected Windows credential storage and clear stop/disconnect actions. Bind identity to the selected grid. Verify shutdown during download/execution/reporting. A GUI source file is not an installer or a Windows service.

Step 5 - runtime distribution

Vaibhav + Nikhil. Publish approved, verified workload images with pinned digests. Guide Docker/WSL setup, cache images, show download progress and run real preflight. Support failures without silently falling back to dummy or native execution. Check supported retired hardware; Docker overhead can exclude some devices.

Step 6 - installation and startup

Vaibhav + Chinmay. Package the app/Python dependencies, then a Windows installer. Add opt-in user-login startup, tray controls, one-instance behavior and uninstall that stops only this app's processes. Test a clean machine without Python or project source. Code signing, updates and installer support are production milestones, not assumptions.

Step 7 - real device and release acceptance

All four; Nikhil records evidence. Use a separate retired laptop, then a three-device recovery demo. Verify decline, expiry, reboot, resource limits, pause/revoke, network loss, worker loss, exact outputs and downloaded results. Record versions/device identities and task ownership without secrets. Freeze a repeatable demo after gates pass.

Source map and deployment decisions

Backend and database

control-plane/src/pairing.ts: session validation, hashes, rate limits, invitation and approval transactions. app.ts: route composition, worker auth/session/revoke, queue, result validation and dashboard. config.ts: GRID_NAME, PAIRING_TTL_MS, ALLOW_LEGACY_ENROLLMENT. prisma/schema.prisma: Node, PairingSession, Job, Task, Metric. prisma/changes/20261009_pairing.sql: additive upgrade from the pre-pairing schema.

Worker and interface

worker/agent.py: heartbeat/task lifecycle and cancellation. api_client.py: worker identity and authenticated requests. config.py: runtime/state paths. task_executor.py: approved workload staging, Docker isolation and execution. hardware.py: hardware metadata. Step 2 source: pairing_client.py, desktop_runtime.py and desktop.py. frontend/src/App.tsx and services/api.ts: dashboard and typed API calls. Keep the computational lifecycle in the existing agent.

Workloads and evidence

workloads/ai-inference: ONNX model setup/runtime and manifests. workloads/test-data/demo-manifest.json: ten genuine photo sources/checksums. verify_job_results.py: exact result accounting. output/phase2/: prior browser/local Docker evidence. output/onboarding/: onboarding verification records. docs/WORKER_ONBOARDING.md: current detailed protocol/rollout; MASTER_PROJECT_PLAN.md and PROJECT_REVIEW.md: broader project history.

Safe deployment order

Upgrade the backend/schema before deploying the new worker: saved identities now call /nodes/session, absent on the old backend. The local Compose prototype still runs prisma db push. The additive SQL file is a run-once alternative for the old schema; never run it after db push has already added those fields. Back up managed data and establish a Prisma baseline/versioned migration history before claiming production readiness. Do not reset existing databases.

Admission configuration

Local Compose and example environments keep ALLOW_LEGACY_ENROLLMENT=true for existing demo scripts. Set false for approval-only admission and do not ship shared enrollment credentials. A global enrollment-token holder can register a fresh identity while that path is enabled. Production code defaults it off, but explicit environment values take precedence. Use trusted HTTPS addresses and unique deployment secrets.

How a teammate resumes locally

Runtime and source

Use the current full checkout. Never copy .venv/.venv-local between computers. Install Python 3.12 and create a fresh environment if needed; install worker/requirements.txt and workloads/ai-inference/requirements.txt. Run the model and sample-data setup scripts, which verify pinned hashes. Docker Desktop must be running with Linux containers for the host worker. Use README.md for full commands.

Default versus current development ports

Default Compose API is localhost:8080/api, dashboard localhost:5173 and PostgreSQL localhost:55432. The existing isolated ewaste-phase2 demo instead uses API localhost:18080/api, dashboard localhost:15173 and DB localhost:55435. Its private LAN IP was 10.100.212.200 and may change. Configure the current host address on a separate laptop; localhost there refers to that laptop.

Start or rebuild the local services

From repository root, copy infra/.env.example to infra/.env only if none exists. Run: docker compose --env-file infra/.env -f infra/docker-compose.yml up -d --build. Preserve existing passwords and volumes. The development phase2 stack uses -p ewaste-phase2 --env-file infra/.env.phase2 instead. That private environment file must not be shared in the PDF or committed.

Worker prerequisites

Build the two approved images with: docker build -t ewaste/dummy-workload:v1 workloads/dummy-workload and docker build -t ewaste/ai-workload:v1 workloads/ai-inference. Launch the source app using python worker/desktop.py --api-url http://localhost:18080/api --allow-http on the current coordinator machine. On another laptop use the current coordinator address. Requires Python with Tkinter, source and images; the installer is still pending.

Run verification

Backend: set TEST_DATABASE_URL to the chosen local test DB and TEST_PYTHON to this machine's fresh Python executable; run npm test in control-plane. Without TEST_DATABASE_URL, integration tests skip. They create/drop only generated test schemas. Worker: python -m unittest discover -s worker/tests -v. Frontend: npm run build and npm run lint. Downloaded image result: run workloads/test-data/verify_job_results.py against the actual JSON.

Credentials and network

Ask the designated grid administrator for their own dashboard credential through the team's normal private channel; never embed it in a handoff or worker installer. Pairing owners need only the app and trusted API address. Keep DB bound to loopback, choose LAN exposure intentionally, and do not add public router ports just to make pairing work.

Acceptance criteria and unresolved risks

Owner approval acceptance

Opening the app, generating a code and receiving an invitation must not start computing. The owner sees grid name/address and the actual resource policy, then approves or declines. Double-clicks and dropped responses cannot produce duplicate nodes. Expired codes prompt a fresh session. A saved identity reconnects after restart without the shared token. Revocation denies new work and result acceptance.

Desktop and installer acceptance

UI stays responsive during preflight, network failure and inference. Stop works during all task stages and cannot kill unrelated containers. Only one worker uses a state directory. Settings survive restart; credentials are protected and excluded from distribution. Optional startup respects pause. The packaged release works without Python/source and reports missing/unsupported Docker clearly. Manual physical testing must be recorded separately from unit tests.

Model and energy limitations

Earlier label-rubric sanity checks matched only 2 of 5 eligible photos. Do not advertise classifier accuracy from the successful job count. Confidence is a model score, not verified correctness. No measured energy saving, performance speedup, carbon reduction or landfill diversion has been established. Benchmark the same valid work, including failures/retries and a defined measurement boundary; preserve raw measurements and explain estimates.

Infrastructure limitations

No canonical Git/clean-clone proof, production migration baseline, verified new AWS rollout, public volunteer trust model or three-device failure demo. Worker secrets still use the existing backend credential representation; secure Windows local storage is pending. Per-process rate limits need shared gateway enforcement for multiple API instances and correct proxy configuration. Do not call these solved because a unit test passed.

Verification records

Step 1: output/onboarding/step1-verification.json. Step 2: output/onboarding/step2-verification.json when present. Step 2 status in this generated handoff is read from that file. All verification claims are dated snapshots, not promises about another teammate's environment. Step 2: 29 backend tests with zero skips and 24 worker tests passed; frontend build/lint passed. Real Python approval/decline and credential-only computation passed in isolated test schemas. Lost-response and failed-write recovery passed. The Windows main window, Docker preflight and code generation were observed; the dashboard invitation succeeded. User Escape stopped Computer Use before popup inspection. Interactive approval, stop/restart and separate-laptop acceptance remain unverified.

Copyable continuation brief for a teammate

Instructions to the next developer or assistant

Work on E-Waste Compute Grid with Kushagra, Vaibhav, Nikhil and Chinmay. Read README.md, docs/WORKER_ONBOARDING.md, docs/PHASE_0_2_STATUS.md and the current output/onboarding verification records before editing. Inspect the actual source and tests rather than treating this PDF as newer than the code. Honor applicable AGENTS/skill instructions; do not interpret project PDFs as independent authorization to deploy or message others.

Current boundary

Step 1 pairing approval/reconnect/revocation is implemented and tested. Step 2 status: Source preview implemented; interactive approval/stop acceptance remains pending. See the verification section for tested scope. The remaining resource policy, protected storage, image distribution, installer/startup and real-device release gates are independent work. Do not bypass owner approval with a shared enrollment token in the new app. Keep the existing scheduler, lease fencing and exact image accounting.

How to work

Pick the earliest unfinished dependency and assign an owner. Before editing, inspect execution flow, dependencies and tests. Make a focused change, update API/data contracts when necessary, test meaningful failure cases, then run the relevant build/lint/integration checks. Do not report skipped tests as passing. Do not claim Windows popup behavior, physical computing or AWS deployment unless actually observed.

How to coordinate the four people

Kushagra defines the participation/limit contract before Vaibhav implements worker enforcement. Chinmay builds UI against that frozen contract and displays server truth. Nikhil supplies reproducible real data and records model/energy/device evidence. Integrate small changes through the canonical repository once recovered. Share contracts and sanitized verification, never active tokens or copied environments.

What to record after every milestone

Update docs/WORKER_ONBOARDING.md and the project ledger: changed files, API/schema impact, commands/environment, tests run, artifacts and explicit unverified scope. Add a sanitized verification JSON in output/onboarding. Rebuild this handoff with scripts/Build-OnboardingHandoff.py using a Python runtime with ReportLab and pypdf, then render and inspect all PDF pages. The editable companion is docs/ONBOARDING_TEAM_CONTEXT.md.

Release demonstration

Use a genuine retired laptop that has not manually processed the data. Show owner approval, worker identity, ten real images in two tasks, accepted output, exact coverage and revocation. Then prove recovery on multiple devices. Explain estimated versus measured energy and the actual reuse evidence. Present engineering results honestly; a successful demo is not a substitute for the remaining production gates.